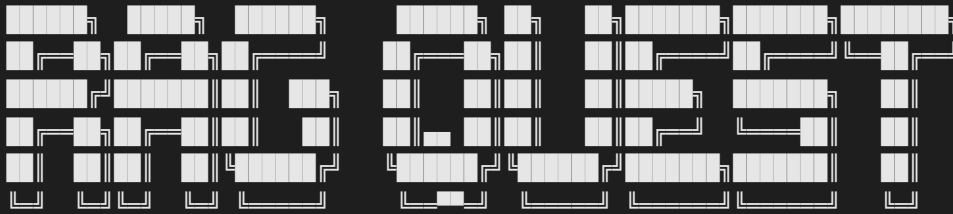




LOCAL RAG ASSISTANT — Beginner Roadmap



offline Q&A chatbot • Microsoft Foundry Local • zero internet

Your mentor's note (read once): You started July 1, it's July 16, you've done nothing yet. **That is fine.** This roadmap compresses the 5-week plan into ~12 focused days. You have 2 weeks full-time — that's enough. We go **one small step at a time**. Every step tells you exactly what to do, why, and how to check it worked. You type the code, you learn by doing, and you **ask me every question**. Nothing here is a race. Finish a step → tick the box → commit → next step.



HOW TO USE THIS DOCUMENT

Every step has the **same 5 parts**, always in the same order. Your brain will learn the rhythm and stop having to re-orient:

Icon	Means
 Goal	What this step achieves, in one sentence
 Why	The reason it matters (so it's not just magic)
 Do	The exact commands/code to type
 Check	How you know it worked before moving on
 Commit	The git command to save your progress

Rules for you:

-  Do steps **in order**. Don't skip ahead. Each one builds on the last.

- ☒ One step per sitting is totally fine. Close the laptop after a green ☒.
- ☒ If a ☒ check fails → **stop, don't continue, ask me.** Paste the error.
- ☒ See ☐ Ask me: prompts? Those are the exact questions worth asking when curious.
- ☒ ☒ YOU WRITE THIS = code I want *you* to write yourself (I'll give hints, not the answer). That's where the learning happens.

Progress tracker — tick these as you finish each day:

```

PHASE 0 • SETUP      [ ] Day 1
PHASE 1 • FOUNDATIONS [ ] Day 2 [ ] Day 3 [ ] Day 4
PHASE 2 • BUILD      [ ] Day 5 [ ] Day 6 [ ] Day 7 [ ] Day 8
PHASE 3 • POLISH     [ ] Day 9 [ ] Day 10 [ ] Day 11 [ ] Day 12

```

THE BIG PICTURE (understand this before you build anything)

Your app does **one loop**. Memorize it — everything you build is a piece of this:

```

YOU type a question
|
v
[1] EMBED the question → turn text into a list of numbers
                        (a "vector")
|
v
[2] SEARCH the database → find chunks whose vectors are
                        closest = most similar in meaning
|
v
[3] AUGMENT the prompt → glue those chunks onto your question
|
v
[4] GENERATE the answer → local LLM reads chunks + question,
                        writes an answer from YOUR docs
|
v
ANSWER appears (with source), all offline

```

That's **RAG**: **R**etrieve, **A**ugment, **G**enerate. Before you can do that loop live, you first have to **load your documents into the database** (that's called *ingestion*, you do it once up front).

The 4 layers (clean architecture = keep these separate, never mix them):

Layer	Job	Your files
 App / UI	Talk to the human	<code>app.py</code> (Streamlit), <code>main.py</code> (CLI)
 Logic / Pipeline	Orchestrate the RAG loop	<code>src/retriever.py</code> , <code>src/generator.py</code> , <code>src/ingest.py</code>
 Data	Store & fetch chunks + vectors	<code>src/database.py</code> (SQLite)
 AI	Embeddings + local LLM	<code>src/embeddings.py</code> , Foundry Local

Golden rule of clean architecture: the UI never talks to the database directly. It calls a logic function, which calls the data layer. Each layer only knows about the one below it. This is why your code won't turn into spaghetti.

TECH STACK (what each tool is and why we picked it)

Tool	What it is	Why this one
Python 3.11+	Programming language	You have it; the AI world runs on it
Git + GitHub	Save history / backup / show your work	Industry standard; you'll commit constantly
Foundry Local	Runs an LLM <i>on your laptop</i> , offline	The whole point of the project — no cloud, no API key
sentence-transformers	Turns text → vectors (embeddings)	Simplest reliable offline embedder for beginners
SQLite	A database that's just one file	Built into Python, zero setup, perfect for local
NumPy	Fast math on lists of numbers	Cosine similarity in one line
Streamlit	Build a web UI with pure Python	You wanted a UI + retro look, this is the fastest path
VS Code	Your code editor	You have it

 **A choice I made for you & why:** The official plan says use Foundry Local for *both* embeddings and chat. Foundry's embedding API is still shifting and it trips up beginners. So

we use `sentence-transformers` for embeddings (rock solid, offline) and Foundry Local only for the chat LLM. Same result, far fewer tears. If you want to swap to Foundry embeddings later, that's a clean one-file change — because clean architecture.

PHASE 0 — SETUP (Day 1)

Goal of today: a real project folder, a git repo, a Python environment, a first commit pushed to GitHub, **and** Foundry Local installed with a model saying hello. When today's green, you have a professional skeleton. No RAG yet — just foundations that won't wobble.

Step 1 — Create the project folder

 **Goal:** A clean home for all your code, on your Desktop, separate from this plan folder.

 **Why:** Keep the *plan* (this folder) and the *project* (your code) apart. Mixing docs and code is the first mess to avoid.

 **Do** — open **PowerShell** (Start menu → type "PowerShell" → Enter) and run:

```
cd $HOME\Desktop
mkdir local-rag-assistant
cd local-rag-assistant
```

 **Check:** Run `pwd`. It should end in `...\Desktop\local-rag-assistant`.

 **Ask me:** "What does `cd` and `$HOME` mean?" if any command is unfamiliar. Never run a command you don't understand — ask first, always.

Step 2 — Turn it into a Git repository

 **Goal:** Start tracking history from the very first line of code.

 **Why:** Git is a **time machine + save points**. Every commit is a save you can return to. You start git *now*, empty, so it captures everything from the beginning.

 **Do:**

```
git init
git branch -M main
```

Then tell git who you are (only needed once per computer — skip if you've done it before):

```
git config --global user.name "Your Name"
git config --global user.email "baranoral88@gmail.com"
```

✅ **Check:** `git status` prints `On branch main` and `No commits yet`.

 **Git vocabulary you now own:**

- **repository (repo)** = the folder git is watching
- **branch** = a parallel line of work; `main` is the trunk
- **commit** = one save point with a message

Step 3 — Create the folder skeleton

 **Goal:** Build the empty clean-architecture structure *before* writing logic.

 **Why:** Deciding where things go up front means you never have to ask "where does this file belong?" mid-code. The structure *is* the plan.

 **Do:**

```
mkdir src, data, data\docs, tests
New-Item src\__init__.py, tests\__init__.py, README.md, requirements.txt, .gitignore
```

Your folder is now:

```
local-rag-assistant/
├── .gitignore      ← tells git what to NOT track
├── README.md       ← project front page (write it last)
├── requirements.txt ← list of Python packages you use
├── data/
│   └── docs/       ← your source documents go here
└── src/            ← all your logic + data + AI code
```

```
|   └─ __init__.py    ← makes 'src' an importable Python package
|   └─ tests/
|       └─ __init__.py
```

 `__init__.py` is an empty file that tells Python "this folder is a package you can import from." That's it. It stays empty.

✅ **Check:** In VS Code, `code .` (opens this folder). You see the tree above in the sidebar.

Step 4 — Write `.gitignore` (do this BEFORE your first commit)

 **Goal:** Stop git from tracking junk (the virtual environment, caches, the database file).

 **Why:** Some files are huge, machine-specific, or regenerated. Committing them pollutes history. `.gitignore` is the bouncer at the door.

 **Do:** Open `.gitignore` in VS Code and paste:

```
# Python virtual environment
.venv/
__pycache__/
*.pyc

# The database is generated by our ingest script – don't track it
data/*.db

# Secrets & OS noise
.env
.DS_Store
Thumbs.db

# Editor
.vscode/
```

✅ **Check:** The file is saved. (You'll confirm it works after the venv exists.)

Step 5 — Create a Python virtual environment

 **Goal:** An isolated Python sandbox just for this project's packages.

 **Why:** A **virtual environment (venv)** keeps *this* project's packages separate from every other project and from your system Python. No version clashes, ever. Pros always use one.

 **Do:**

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

Your prompt should now start with `(.venv)`. That means the sandbox is **active**.

 **If PowerShell blocks the activation** with a "running scripts is disabled" error, run this once, then retry the activate line:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

 **Ask me** to explain what that does before you run it if you're unsure — it's safe, but understanding beats blindly pasting.

 **Check:** Prompt shows `(.venv)`. Run `python --version` → 3.11 or higher.

 **You'll re-activate every session.** New PowerShell window = run `.\.venv\Scripts\Activate.ps1` again first. If you forget, packages "vanish" — that's the #1 beginner confusion. It's not vanished, the sandbox is just closed.

Step 6 — Your first commit

 **Goal:** Save this skeleton as save-point #1.

 **Why:** Commit early, commit often. A tiny first commit proves the whole git pipeline works before you have anything to lose.

 **Do:**

```
git add .
git status
```

Look at `git status` output. You should see your files listed as "to be committed" — and you should **NOT** see `.venv/` (because `.gitignore` is doing its job).

✅ **This is your `.gitignore` check:** if `.venv/` shows up, stop and ask me — the ignore file isn't working.

Now commit:

```
git commit -m "chore: initial project skeleton"
```

🗨️ **About commit messages** — we use **Conventional Commits**. Format: `type: description`.

Type	Use when
feat	You added a new feature
fix	You fixed a bug
docs	Docs only (README, comments)
chore	Setup, config, housekeeping
refactor	Reshaped code without changing behavior
test	Added or changed tests

Keep the description short, lowercase, present tense: "add retriever", not "Added the retriever function".

✅ **Check:** `git log --oneline` shows one line — your commit.

Step 7 — Push to GitHub

🎯 **Goal:** Your code lives safely in the cloud and you can show it off.

💡 **Why:** GitHub = backup + portfolio + proof of work. If your laptop dies, your project doesn't.

📝 **Do:**

1. Go to **github.com** → click + (top right) → **New repository**.

2. Name it `local-rag-assistant` . **Leave everything else unchecked** (no README, no .gitignore — you already have them). Click **Create repository**.
3. GitHub shows you commands. Use the **"...or push an existing repository"** block. It looks like:

```
git remote add origin https://github.com/YOUR-USERNAME/local-rag-assistant.git
git push -u origin main
```

(It may pop a browser to log in — approve it.)

✅ **Check:** Refresh the GitHub page. Your files are there.

📖 **New vocabulary:**

- **remote** = a copy of your repo on another machine (GitHub). `origin` is its nickname.
- **push** = upload your commits to the remote.
- `-u origin main` = "remember this remote+branch, so next time I just type `git push`".

Step 8 — Install Foundry Local (the offline AI engine)

🎯 **Goal:** Get Microsoft's local LLM runtime onto your machine and prove it runs a model.

💡 **Why:** This is the heart of "offline AI." Foundry Local downloads and runs language models directly on your laptop's CPU/GPU/NPU. No cloud, no account, no API key.

📄 **Do** — in PowerShell:

```
winget install Microsoft.FoundryLocal
```

Close and reopen PowerShell after it finishes (so the new `foundry` command is found). Then verify + run a small model:

```
foundry --version
foundry model list
foundry run qwen2.5-0.5b
```

⚠️ **CLI note (verified on v0.10.2):** the command is `foundry run <model>` — NOT `foundry model run`. In this version `run / chat / complete` are top-level commands; `model` is only for

list/download/load. If a command is rejected, run `foundry --help` — your own binary is always the source of truth over any tutorial.

That last command **downloads** a small model (a few GB — needs internet *this once*), loads it, and drops you into a **live chat in your terminal**. Ask it something:

```
Interactive Chat. Enter /? for help.  
> Hello, who are you?
```

Type `/exit` or `Ctrl+C` to leave.

✅ **Check:** The model replied to you in the terminal. **You just ran an LLM with no internet dependency after download.** That's the milestone.

⚠️ **Foundry Local needs a fair bit of disk + RAM.** If `winget` fails or the model won't load, **stop and paste me the exact error.** Common fixes are disk space, or picking an even smaller model — we'll sort it together.

❓ **Ask me:** "What's the difference between `qwen2.5-0.5b` and other models?" and "How do I pick a model size for my laptop?" — good questions to understand now.

🚩 Day 1 done. Milestone:

- ✅ Git repo on GitHub with a clean skeleton
- ✅ Working Python venv
- ✅ Foundry Local installed, a model chatted with you offline

💾 Nothing to commit for the Foundry step (it's system software, not your code). Just make sure Steps 1–7 are pushed. **Close the laptop. Real progress today.**

🟢 PHASE 1 — FOUNDATIONS (Days 2–4)

Now you learn the 3 core skills the RAG loop needs, each in a **throwaway practice script** before it goes into real code: (Day 2) call the LLM from Python, (Day 3) embeddings + similarity, (Day 4) SQLite. You're building intuition, not the final app yet.

 **Where practice scripts go:** make a folder `scratch/` for these. They're for learning, not part of the app. Add `scratch/` to `.gitignore` — or commit them, your call. I'd commit them so future-you can see the learning journey.

Step 9 — Install your first Python packages

 **Goal:** Get the Foundry Local Python SDK. It ships **its own chat client** — no separate OpenAI package needed. Microsoft's SDK talks to the local model directly.

 **Why:** `requirements.txt` is your project's shopping list of packages. Anyone (incl. future-you on a new laptop) can recreate your exact setup from it.

 **Do** — make sure `(.venv)` is active, then:

```
pip install foundry-local-sdk
```

Now **freeze** what you installed into the shopping list:

```
pip freeze > requirements.txt
```

✓ **Check:** Open `requirements.txt` — it lists `foundry-local-sdk` and its dependencies with version numbers.

 **Commit:**

```
git add requirements.txt
git commit -m "chore: add foundry-local-sdk dep"
```

Step 10 — Call the local LLM from Python (Day 2 core)

 **Goal:** Do in *code* what you did in the terminal at Step 8 — get an answer from the local model programmatically.

 **Why:** Your app can't type into a terminal chat. It needs to call the model from Python and get text back. This is that skill, isolated.

 **File:** `scratch/hello_llm.py`

 **Do:** The Foundry SDK starts the local model **and** gives you its own chat client — no OpenAI package involved. Here's the **complete** starter (type it, don't paste-and-forget — typing builds memory):

```
# scratch/hello_llm.py
# Goal: ask the local model one question from Python.

from foundry_local_sdk import Configuration, FoundryLocalManager

MODEL_ALIAS = "qwen2.5-0.5b"

# Start the Foundry Local service for our app.
config = Configuration(app_name="rag_assistant")
FoundryLocalManager.initialize(config)
manager = FoundryLocalManager.instance

# Register the execution providers (CPU/GPU backends) once.
manager.download_and_register_eps()

# Fetch, download, and load the model into memory.
model = manager.catalog.get_model(MODEL_ALIAS)
model.download()
model.load()

# The SDK hands us its own chat client – no OpenAI client needed.
client = model.get_chat_client()
messages = [
    {"role": "system", "content": "You are a helpful, concise assistant."},
    {"role": "user", "content": "Explain what RAG is in two sentences."},
]

# Stream the answer piece by piece as the model generates it.
print("Assistant: ", end="", flush=True)
for chunk in client.complete_streaming_chat(messages):
    if chunk.choices:
        piece = chunk.choices[0].delta.content
        if piece:
            print(piece, end="", flush=True)
print()

model.unload() # free the memory when done
```

Run it:

```
python scratch\hello_llm.py
```

⚠️ **The Foundry SDK's exact class/method names are still changing between versions.** If you get an `ImportError` or `AttributeError`, **do not fight it alone — paste me the error.** I'll match the code to *your* installed version in seconds. This is expected, not your fault.

✅ **Check:** The script prints a 2-sentence explanation of RAG.

📺 **What you just learned:**

- **system message** = standing instructions for the model ("be concise")
- **user message** = the actual question
- `chunk.choices[0].delta.content` = each small piece of the answer as it streams in (a non-streaming call would give one `message.content` instead)

❓ **Ask me:** "What's `temperature`?" and "What are tokens?" — you'll want both soon.

💾 **Commit:** `git add scratch/hello_llm.py && git commit -m "feat: call local LLM from python"`

Step 11 — Embeddings & similarity (Day 3 core)

🎯 **Goal:** Turn sentences into vectors and measure which are closest in meaning.

💡 **Why:** This is **step [1] and [2]** of the RAG loop — the "search by meaning" magic. An embedding model maps text → a list of ~384 numbers where *similar meaning* = *similar numbers*. "Cosine similarity" scores how close two vectors point (1.0 = identical meaning, 0 = unrelated).

📦 **Install the embedder** (downloads a ~90MB model once, then offline forever):

```
pip install sentence-transformers numpy
pip freeze > requirements.txt
```

📁 **File:** `scratch/hello_embeddings.py`

 I'll give you the setup. **You write the similarity comparison** — that's the learning bit.

```
# scratch/hello_embeddings.py
from sentence_transformers import SentenceTransformer
import numpy as np

model = SentenceTransformer("all-MiniLM-L6-v2")

sentences = [
    "The cat sat on the mat.",
    "A feline rested on the rug.",
    "Python is a programming language.",
]

# Each sentence becomes a 384-number vector.
vectors = model.encode(sentences)
print("Vector shape:", vectors.shape)  # (3, 384)

def cosine_similarity(a, b):
    """Return how similar two vectors are, from -1 to 1."""
    # 🌸 YOU WRITE THIS (2-3 lines).
    # Hint: cosine similarity = dot(a, b) / (norm(a) * norm(b))
    # NumPy gives you np.dot(a, b) and np.linalg.norm(a).
    pass

# Compare sentence 0 to sentences 1 and 2:
print("cat vs feline :", cosine_similarity(vectors[0], vectors[1]))  # expect HIGH
print("cat vs python :", cosine_similarity(vectors[0], vectors[2]))  # expect LOW
```

🌸 **YOU WRITE THIS:** the body of `cosine_similarity`. Try it yourself first. Stuck after a real attempt? Ask me and show me what you tried.

✅ **Check:** "cat vs feline" scores clearly higher (~0.6+) than "cat vs python" (~0.1). That's semantic search working: the machine sees "cat/feline" mean nearly the same thing even with zero shared words.

❓ **Ask me:** "Why 384 numbers? What do they represent?" — great conceptual question.

 **Commit:** `git commit -am "feat: embeddings + cosine similarity demo"` (-am = add tracked changes *and* commit in one go. Only works on files git already tracks.)

Step 12 — SQLite basics (Day 4 core)

 **Goal:** Create a database file, put rows in, get rows out — with plain Python.

 **Why:** This is your **data layer**. You'll store each document chunk + its vector here so you don't recompute embeddings every run. `sqlite3` is built into Python — nothing to install.

 **File:** `scratch/hello_sqlite.py`

 **Do** — type this and run it:

```
# scratch/hello_sqlite.py
import sqlite3

# Connect (creates the file if it doesn't exist).
conn = sqlite3.connect("scratch/practice.db")
cur = conn.cursor()

# Create a table. Do this once; IF NOT EXISTS makes it safe to re-run.
cur.execute("""
    CREATE TABLE IF NOT EXISTS notes (
        id      INTEGER PRIMARY KEY AUTOINCREMENT,
        content TEXT NOT NULL
    )
""")

# Insert rows. The ? is a placeholder – ALWAYS use it, never f-strings in SQL.
cur.execute("INSERT INTO notes (content) VALUES (?)", ("Buy milk",))
cur.execute("INSERT INTO notes (content) VALUES (?)", ("Learn RAG",))
conn.commit() # commit = actually save to disk

# Read rows back.
cur.execute("SELECT id, content FROM notes")
for row in cur.fetchall():
    print(row)

conn.close()
```

Run it twice:

```
python scratch\hello_sqlite.py
```

✅ **Check:** First run prints 2 rows. Second run prints 4 (it inserted again). You now see how a database *persists* between runs.

⚠️ **Security habit — the ? placeholder:** never build SQL by gluing strings (`"... VALUES ('" + text + "')`). That's the classic **SQL injection** bug. Always pass values as the `?` tuple

like above. This matters even in a local app — build the safe habit now.

? **Ask me:** "How will I store a *vector* (list of numbers) in a TEXT column?" — this is exactly the problem you solve in Phase 2. Think about it now; the answer is "as JSON".

 **Commit:** `git commit -am "feat: sqlite create/insert/select demo"`

🏁 Phase 1 done. Milestone:

You can, in isolation: **call the local LLM**, **embed text & score similarity**, and **store/fetch from SQLite**. These three are every ingredient of RAG. Next phase glues them into the real app.

 **Optional: learn one git branch move before Phase 2.** So far you've committed straight to `main`. Pros build features on a **branch**, then merge. Try it once:

```
git checkout -b practice-branch    # create + switch to a new branch
# ...make a tiny change, commit it...
git checkout main                  # switch back
git merge practice-branch          # bring the change into main
git branch -d practice-branch      # delete the finished branch
```

? **Ask me** to walk through what each line did. In Phase 2 you'll use a branch per feature for real.

PHASE 2 — BUILD THE APP (Days 5–8)

Now you build the real thing, one module per file, following the clean architecture. The **workflow for every feature from here on:**

```
1. git checkout -b feature/NAME    ← branch off main
2. write the code                   ← one file, one job
3. run it, make it work
4. git commit -am "feat: ..."    ← save
```



```
5. git checkout main && git merge feature/NAME
6. git push
```

This is how real teams work. You'll internalize it by Day 8.

Step 13 — Central config (src/config.py)

 **Goal:** One place for every setting: model names, file paths, how many chunks to retrieve.

 **Why:** When a value (like "which model") appears in 5 files, changing it is a nightmare. Put it **once** in config; everything imports from there. This is the single most-loved habit in clean code.

 **File:** src/config.py

```
git checkout -b feature/config
```

```
# src/config.py
"""All project settings live here. Import from this file, never hardcode."""
from pathlib import Path

# Folders (built relative to the project root, so it works on any machine).
ROOT_DIR = Path(__file__).resolve().parent.parent
DATA_DIR = ROOT_DIR / "data"
DOCS_DIR = DATA_DIR / "docs"
DB_PATH = DATA_DIR / "knowledge.db"

# Models
EMBED_MODEL = "all-MiniLM-L6-v2" # sentence-transformers embedder
LLM_ALIAS = "qwen2.5-0.5b" # Foundry Local chat model

# Retrieval settings
TOP_K = 3 # how many chunks to feed the LLM
CHUNK_MAX_CHARS = 800 # rough size of each document chunk
```

 **Check:** `python -c "from src.config import DB_PATH; print(DB_PATH)"` prints the full path to `knowledge.db`. (Run this from the project root.)

 **Commit & merge:**

```
git commit -am "feat: central config module"
git checkout main && git merge feature/config && git push
```

Step 14 — Data layer (src/database.py)

🔗 **Goal:** All SQLite code lives in one module with clean functions: `init_db()`, `insert_chunk()`, `get_all_chunks()`.

💡 **Why:** Nobody else in your app should write SQL. They call these functions. If you ever swap SQLite for something else, you change *only this file*. That's the clean-architecture payoff.

📁 **File:** `src/database.py` · **Branch:** `git checkout -b feature/database`

I'll give you the skeleton. **You write** `get_all_chunks()` — you've done SELECT already.

```
# src/database.py
"""Data layer: the ONLY place that talks to SQLite."""
import json
import sqlite3
from src.config import DB_PATH

def _connect():
    return sqlite3.connect(DB_PATH)

def init_db():
    """Create the chunks table if it doesn't exist."""
    with _connect() as conn:
        conn.execute("""
            CREATE TABLE IF NOT EXISTS chunks (
                id            INTEGER PRIMARY KEY AUTOINCREMENT,
                source         TEXT NOT NULL,           -- which document this came from
                content        TEXT NOT NULL,           -- the chunk text
                embedding       TEXT NOT NULL            -- the vector, JSON-encoded
            )
        """)

def insert_chunk(source: str, content: str, embedding: list[float]):
    """Save one chunk + its vector. The vector becomes JSON text."""
    with _connect() as conn:
        conn.execute(
            "INSERT INTO chunks (source, content, embedding) VALUES (?, ?, ?)",
            (source, content, json.dumps(embedding)),
        )

def get_all_chunks() -> list[dict]:
    """Return every chunk as a dict: {source, content, embedding(list)}."""
    # 🌱 YOU WRITE THIS.
    # Steps:
```

```
# 1. connect, SELECT source, content, embedding FROM chunks
# 2. for each row, json.loads() the embedding back into a list
# 3. return a list of dicts like {"source":..., "content":..., "embedding":[...] }
pass
```

✿ **YOU WRITE:** `get_all_chunks()`. Hint: `json.loads(row[2])` turns the stored text back into a Python list.

✓ **Check:** write a 3-line test in `scratch/` that calls `init_db()`, `insert_chunk("t","hi",[0.1,0.2])`, then prints `get_all_chunks()`. You should see your dict.

❓ **Ask me:** "Why store the vector as JSON instead of its own columns?" — the answer teaches you a real trade-off.

💾 `git commit -am "feat: sqlite data layer" && git checkout main && git merge feature/database && git push`

Step 15 — AI layer (`src/embeddings.py`)

🎯 **Goal:** Two clean functions: `embed_text(str) -> list[float]` and `embed_batch(list[str]) -> list[list[float]]`.

💡 **Why:** Loading the embedding model is slow (~2s). You want to load it **once** and reuse it. This module hides that detail behind two simple functions.

📁 **File:** `src/embeddings.py` · **Branch:** `feature/embeddings`

```
# src/embeddings.py
"""AI layer: text -> vectors. Loads the model once, reuses it."""
from sentence_transformers import SentenceTransformer
from src.config import EMBED_MODEL

_model = None # module-level cache

def _get_model():
    global _model
    if _model is None:
        _model = SentenceTransformer(EMBED_MODEL) # loaded only on first call
    return _model
```

```
def embed_text(text: str) -> list[float]:
    """One string -> one vector (as a plain Python list)."""
    return _get_model().encode(text).tolist()

def embed_batch(texts: list[str]) -> list[list[float]]:
    """Many strings -> many vectors. Faster than looping embed_text."""
    return _get_model().encode(texts).tolist()
```

✓ **Check:** `python -c "from src.embeddings import embed_text; print(len(embed_text('hi')))"` prints 384.

💡 **The `_model = None` pattern** is called *lazy loading*: don't do expensive work until the first time it's actually needed, then remember the result. You'll see it everywhere.

💾 Commit & merge (same rhythm as before).

Step 16 — Chunking (src/chunking.py)

🎯 **Goal:** A function `chunk_text(str) -> list[str]` that splits a long document into passage-sized pieces.

💡 **Why:** You can't embed a whole 10-page doc as one vector — meaning gets blurred, and the LLM has a context limit. RAG works on **chunks** (~1–3 paragraphs). Good chunking is *the* thing that most affects answer quality. (Remember that for your final presentation's "lessons learned"!)

📁 **File:** `src/chunking.py` · **Branch:** `feature/chunking`

🌱 **This one is mostly YOURS to write** — it's simple string work and a great exercise.

```
# src/chunking.py
"""Split documents into passage-sized chunks."""
from src.config import CHUNK_MAX_CHARS

def chunk_text(text: str) -> list[str]:
    """Split text into chunks of roughly CHUNK_MAX_CHARS characters.

    Simple strategy (good enough to start):
    1. Split the text into paragraphs (they're separated by blank lines).
    2. Greedily glue paragraphs together until adding the next one would
       exceed CHUNK_MAX_CHARS, then start a new chunk.
    3. Return the list of chunk strings (skip empty ones)."""
```

```
"""
# 🌟 YOU WRITE THIS.
# Hints:
# - paragraphs = text.split("\n\n")
# - build up a 'current' string; when len(current) + len(para) > CHUNK_MAX_CHARS,
#   push 'current' to the result list and start fresh.
# - strip whitespace; don't add empty chunks.
pass
```

✅ **Check:** feed it a 3-paragraph string → get back 1–2 chunks. Feed it a 20-paragraph string → get back several. No chunk wildly over `CHUNK_MAX_CHARS`.

ponytail: naive char-based paragraph chunking. Upgrade to token-aware or sentence-boundary splitting only if answer quality suffers. ← leave a comment like this in your code so you remember it's a deliberate simple choice, not an accident.

💾 Commit & merge.

Step 17 — Ingestion pipeline (src/ingest.py)

🎯 **Goal:** One script that reads every doc in `data/docs/`, chunks it, embeds each chunk, and stores it in SQLite. Run once (or whenever docs change).

💡 **Why:** This assembles Steps 14–16 into the **"load the knowledge base"** step from the big-picture diagram. After this runs, your DB is the searchable brain.

📁 **File:** `src/ingest.py` · **Branch:** `feature/ingest`

First, put 5–10 small `.txt` files in `data/docs/`. Course notes, FAQs, a manual — anything you'll want to ask questions about. (Ask me for sample docs if you don't have any.)

```
# src/ingest.py
"""Read docs -> chunk -> embed -> store. Run this once to build the knowledge base."""
from src.config import DOCS_DIR
from src.database import init_db, insert_chunk
from src.embeddings import embed_batch
from src.chunking import chunk_text

def ingest():
    init_db()
    doc_files = list(DOCS_DIR.glob("*.txt"))
```

```

print(f"Found {len(doc_files)} documents.")

for path in doc_files:
    text = path.read_text(encoding="utf-8")
    chunks = chunk_text(text)
    vectors = embed_batch(chunks)          # embed all chunks of this doc at once
    for chunk, vector in zip(chunks, vectors):
        insert_chunk(source=path.name, content=chunk, embedding=vector)
    print(f" {path.name}: {len(chunks)} chunks stored")

print("Ingestion complete.")

if __name__ == "__main__":
    ingest()

```

 **Run:** `python -m src.ingest`

✅ **Check:** It prints each file and its chunk count, ending with "Ingestion complete." Then verify the DB really has rows — reuse `get_all_chunks()` in a quick scratch print.

❓ **Ask me:** "Why `python -m src.ingest` instead of `python src/ingest.py`?" — the `-m` matters for imports to work. Good thing to understand.

 Commit & merge.

Step 18 — Retriever (src/retriever.py)

🎯 **Goal:** `get_top_chunks(query, k) -> list[dict]` — the search step. Embed the query, compare against every stored vector, return the `k` closest chunks.

💡 **Why:** This is **step [2]** of the RAG loop. For your small dataset, "compare against every vector" (brute force) is perfectly fine and easy to understand.

📁 **File:** `src/retriever.py` · **Branch:** `feature/retriever`

```

# src/retriever.py
"""Logic layer: find the chunks most relevant to a query."""
import numpy as np
from src.config import TOP_K
from src.database import get_all_chunks
from src.embeddings import embed_text

```

```
def _cosine(a, b) -> float:
    a, b = np.array(a), np.array(b)
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

def get_top_chunks(query: str, k: int = TOP_K) -> list[dict]:
    """Return the k chunks whose embeddings are closest to the query."""
    query_vec = embed_text(query)
    chunks = get_all_chunks()

    # 🌱 YOU WRITE THIS core part:
    # 1. for each chunk, compute _cosine(query_vec, chunk["embedding"])
    # 2. attach that score to the chunk (e.g. chunk["score"] = ...)
    # 3. sort chunks by score, highest first
    # 4. return the top k
    pass
```

🌱 **YOU WRITE:** the ranking. Hint: `sorted(chunks, key=lambda c: c["score"], reverse=True)[:k]`.

✅ **Check:** ask a question you *know* one of your docs answers → the top chunk's `content` is clearly the relevant passage. Print the scores too; the top one should be noticeably higher.

ponytail: O(n) brute-force scan over all vectors. Fine for <10k chunks. Swap to a vector index (faiss/sqlite-vec) only if retrieval gets slow.

💾 Commit & merge.

Step 19 — Generator (src/generator.py) — the full RAG loop



🎯 **Goal:** `answer_query(question) -> str`. Retrieve chunks, build a prompt with them, call the local LLM, return the grounded answer. **This is the whole project in one function.**

💡 **Why:** Steps [3] Augment + [4] Generate. The prompt is where you *instruct* the model to only use the provided context and to admit when it doesn't know — the anti-hallucination guardrail.

📁 **File:** `src/generator.py` · **Branch:** `feature/generator`

```
# src/generator.py
"""Logic layer: turn a question + retrieved context into a grounded answer."""
```

```

from foundry_local_sdk import Configuration, FoundryLocalManager
from src.config import LLM_ALIAS
from src.retriever import get_top_chunks

_client = None

def _get_client():
    global _client
    if _client is None:
        config = Configuration(app_name="rag_assistant")
        FoundryLocalManager.initialize(config)
        manager = FoundryLocalManager.instance
        manager.download_and_register_eps()
        model = manager.catalog.get_model(LLM_ALIAS)
        model.download()
        model.load()
        _client = model.get_chat_client() # loaded once, reused every call
    return _client

SYSTEM_PROMPT = (
    "You are a helpful assistant that answers ONLY using the provided context. "
    "If the answer is not in the context, say: \"I don't have that information.\" "
    "Cite the source document name in your answer."
)

def answer_query(question: str) -> str:
    chunks = get_top_chunks(question)
    client = _get_client()

    # 🌱 YOU WRITE the prompt assembly:
    # 1. Build a 'context' string joining each chunk's source + content, e.g.
    #     [source: notes.txt]
    #     <chunk content>
    # 2. Build the user message: the context block, then "Question: {question}"
    # 3. Build messages = [{system: SYSTEM_PROMPT}, {user: <your message>}],
    #     then stream the reply (copy the complete_streaming_chat shape from Step 10).
    # 4. Accumulate each chunk.choices[0].delta.content into a string and return it.
    pass

if __name__ == "__main__":
    print(answer_query("put a question your docs can answer here"))

```

🌱 **YOU WRITE:** the prompt assembly + model call. You have every piece from Step 10 and 18 — this is you connecting them. **This is the moment it all clicks. Take your time.**

 **Run:** `python -m src.generator`

✅ **Check (the big one):** It answers your question **using your document's content**, and mentions the source. Then ask something *not* in your docs → it should say "I don't have that information." If it makes something up, your prompt needs tightening — ask me.

? **Ask me:** "How do I make it stream the answer word-by-word like ChatGPT?" — nice upgrade for the UI later.

💾 Commit & merge. **This is your biggest milestone. Push it and be proud.**

Step 20 — CLI app (`main.py`)

🎯 **Goal:** A loop that asks for a question, prints the answer, repeats. Your first real *app* interface.

💡 **Why:** Before the fancy UI, prove the whole pipeline works from a simple entry point. CLI first = least that can go wrong.

📁 **File:** `main.py` (project root) · **Branch:** `feature/cli`

```
# main.py
"""Command-line interface for the local RAG assistant."""
from src.generator import answer_query

def main():
    print("Local RAG Assistant. Type 'quit' to exit.\n")
    while True:
        question = input("You: ").strip()
        if question.lower() in {"quit", "exit"}:
            break
        if not question:
            continue          # ignore empty input
        print("\nAssistant:", answer_query(question), "\n")

if __name__ == "__main__":
    main()
```

🖨️ **Run:** `python main.py`

✅ **Check:** You hold a multi-question conversation with your docs, from the terminal.

💾 Commit & merge & push.

Step 21 — Retro Streamlit UI (app.py)

 **Goal:** A web interface with a **retro CRT-terminal aesthetic** — green phosphor text, monospace font, dark screen, glow.

 **Why:** You wanted to learn UI + a retro look. Streamlit lets you build a web app in pure Python, and custom CSS gives it the vintage vibe.

 **File:** app.py · **Branch:** feature/ui

 **Install:**

```
pip install streamlit
pip freeze > requirements.txt
```

```
# app.py
"""Retro-terminal web UI for the local RAG assistant."""
import streamlit as st
from src.generator import answer_query

st.set_page_config(page_title="RAG Terminal", page_icon="🖥️", layout="centered")

# --- Retro CRT styling -----
st.markdown("""
<style>
  .stApp {
    background-color: #0a0f0a;
    color: #33ff66;
    font-family: 'Courier New', monospace;
  }
  h1, h2, h3, p, label, .stMarkdown {
    color: #33ff66 !important;
    text-shadow: 0 0 4px #33ff66;
  }
  .stTextInput > div > div > input {
    background-color: #0a0f0a;
    color: #33ff66;
    border: 1px solid #33ff66;
    font-family: 'Courier New', monospace;
  }
  .stButton > button {
    background-color: #0a0f0a;
    color: #33ff66;
    border: 1px solid #33ff66;
    font-family: 'Courier New', monospace;
  }
  .stButton > button:hover {
    background-color: #33ff66;
  }
</style>
""")
```

```

        color: #0a0f0a;
    }
</style>
"""', unsafe_allow_html=True)

# --- App -----
st.title("▀ RAG TERMINAL ▀")
st.caption("offline knowledge assistant · foundry local · no internet")

question = st.text_input("QUERY >", placeholder="ask your documents...")

if st.button("TRANSMIT") and question:
    with st.spinner("retrieving + generating..."):
        answer = answer_query(question)
        st.markdown("**RESPONSE:**")
        st.markdown(answer)

```

 **Run:** `streamlit run app.py` — a browser tab opens automatically.

✓ **Check:** You ask a question in the retro green terminal, hit TRANSMIT, and your grounded answer appears. **You have a real product now.**

 **Make it yours** (great learning, low risk — experiment freely on this branch):

- Add a "scanline" overlay with a CSS `repeating-linear-gradient`.
- Show which source chunks were used (call `get_top_chunks` and list them in an expander).
- Add a blinking cursor, an ASCII logo, a boot-up animation.

? **Ask me** for any of these — the UI is the fun part, and CSS is very "tweak and see."

 Commit & merge & push.

🏁 Phase 2 done. Milestone:

A working offline RAG assistant with a retro web UI. The core project is complete. Everything after this is making it solid and presentable.

PHASE 3 — POLISH, TEST & PRESENT (Days 9–12)

Step 22 — Prompt engineering pass (Day 9)

 **Goal:** Tune `SYSTEM_PROMPT` in `generator.py` until answers are concise, grounded, and honest about not knowing.

 **Why:** The retrieval can be perfect but a sloppy prompt gives sloppy answers. This is cheap, high-impact tuning.

 **Do:** Experiment with the system prompt. Try instructing: answer in ≤ 3 sentences; always name the source; if context is empty, don't guess. Re-run the same test questions after each tweak and compare.

 **Check:** In-docs questions → tight, sourced answers. Out-of-docs questions → clean "I don't have that information." No hallucinations.

ponytail: if you add citation post-processing, keep it minimal — the prompt instruction usually does the job without extra code.

 `git commit -am "fix: tighten system prompt for grounded answers"`

Step 23 — A tiny test (Day 9)

 **Goal:** One runnable check so a future change can't silently break retrieval.

 **Why:** You don't need a big test suite. One smoke test that fails loudly if the core breaks is enough — and it teaches you how tests work.

 **File:** `tests/test_smoke.py`

```
# tests/test_smoke.py
"""Minimal check: cosine similarity behaves, retrieval returns results."""
from src.retriever import _cosine

def test_cosine_identical_is_one():
    assert abs(_cosine([1, 0, 0], [1, 0, 0]) - 1.0) < 1e-6

def test_cosine_orthogonal_is_zero():
    assert abs(_cosine([1, 0], [0, 1])) < 1e-6
```

```
if __name__ == "__main__":
    test_cosine_identical_is_one()
    test_cosine_orthogonal_is_zero()
    print("All smoke tests passed.")
```

 **Run:** `python tests/test_smoke.py`

✓ **Check:** Prints "All smoke tests passed."

 `git commit -am "test: add cosine similarity smoke tests"`

Step 24 — Functional testing (Day 10)

🎯 **Goal:** A table of test questions + whether the assistant handled each correctly.

💡 **Why:** This is your Phase 3 milestone from the official plan, and gold for your presentation.

📝 **Do:** Make a file `TESTLOG.md`. For ~10 questions (some answerable, some not, plus edge cases like empty input and a very vague question), record: question → answer → verdict (✓ correct / ⚠️ weak / ✗ wrong). Fix the worst offenders.

✓ **Check:** `TESTLOG.md` has your results and you've addressed the ✗ rows.

 `git commit -m "docs: add functional test log"`

Step 25 — README & documentation (Day 11)

🎯 **Goal:** A `README.md` that explains the project and lets anyone run it.

💡 **Why:** The README is your project's front door — for graders, for your GitHub portfolio, for future-you. It also proves you understand your own system.

📝 **Do:** Write `README.md` with these sections:

1. **What it is** — one paragraph: offline RAG Q&A assistant with Foundry Local.
2. **Architecture** — paste the big-picture diagram; name the 4 layers.
3. **Setup** — clone, make venv, `pip install -r requirements.txt`, install Foundry Local.
4. **Usage** — `python -m src.ingest` then `streamlit run app.py`.
5. **How it works** — the RAG loop in your own words.

6. **Limitations & choices** — mention the `ponytail`: shortcuts (brute-force search, simple chunking) as *deliberate* decisions. Graders love this.

✅ **Check:** A friend could clone your repo and run it using only the README.

💾 `git commit -m "docs: write project README"`

Step 26 — Cleanup (Day 11)

🎯 **Goal:** Remove debug prints, add a comment to each module's top explaining its job, consistent style.

💡 **Why:** Clean code reads like the roadmap looks. This is the difference between "works" and "professional."

📖 **Do:** Read each file top to bottom. Delete dead code and stray prints. Make sure every `src/` file has a one-line docstring saying what it does (most already do).

💾 `git commit -am "refactor: cleanup and comments"`

Step 27 — Demo prep (Day 12)

🎯 **Goal:** A confident 5-minute demo.

💡 **Why:** The program ends with a presentation. Rehearsed = calm.

📖 **Do — prepare to show:**

1. **Problem:** "General LLMs hallucinate about specific docs. My assistant answers only from a local knowledge base, offline."
2. **Live demo:** ask a question it answers (with source) → ask one it *doesn't* know → it says so. That contrast is the money shot.
3. **How it works:** walk the RAG loop diagram.
4. **Lessons learned:** pick one real thing — e.g. "chunking strategy changed retrieval quality more than anything else."
5. **Retro UI reveal:** the green terminal always lands well.

✅ **Check:** You can do the whole demo without notes, twice.

💾 Final commit + push. **Tag it:**

```
git tag v1.0
git push origin v1.0
```

 A **tag** marks a special commit forever — your v1.0 release.

FINAL FILE STRUCTURE (what you'll have built)

```
local-rag-assistant/
├── .gitignore
├── README.md           ← project front door
├── requirements.txt    ← reproducible setup
├── TESTLOG.md         ← your functional test results
├── main.py            ← CLI entry point
├── app.py             ← retro Streamlit web UI
├── data/
│   ├── docs/          ← your source documents (.txt)
│   └── knowledge.db    ← generated DB (git-ignored)
├── src/
│   ├── __init__.py
│   ├── config.py      ← all settings in one place
│   ├── chunking.py    ← split docs into passages
│   ├── embeddings.py  ← text → vectors (AI layer)
│   ├── database.py    ← SQLite (data layer)
│   ├── ingest.py      ← docs → chunks → vectors → DB
│   ├── retriever.py   ← query → top-k chunks (logic)
│   └── generator.py   ← chunks + question → answer (logic)
└── tests/
    ├── __init__.py
    └── test_smoke.py
```

WHEN YOU GET STUCK (read this before panicking)

1. **Read the error's LAST line first.** It usually names the problem.
2. **Is `(.venv)` active?** Half of all "it broke" is a closed sandbox.
3. **Did you run from the project root?** `python -m src.xxx` needs the root as your folder.
4. **Foundry SDK error?** Expected — the API shifts between versions. **Paste it to me.**
5. **Still stuck?** Bring me: (a) what you ran, (b) the full error, (c) what you expected. That's the perfect question and I'll fix it fast.



GIT COMMANDS CHEAT SHEET (your daily moves)

<code>git status</code>	see what changed
<code>git add .</code>	stage all changes
<code>git commit -m "type: message"</code>	save a checkpoint
<code>git commit -am "type: message"</code>	stage tracked + commit (one shot)
<code>git log --oneline</code>	see your history
<code>git checkout -b feature/name</code>	new branch off current
<code>git checkout main</code>	switch back to main
<code>git merge feature/name</code>	bring a branch into main
<code>git branch -d feature/name</code>	delete a finished branch
<code>git push</code>	upload to GitHub
<code>git tag v1.0</code>	mark a release

Commit types: `feat` `fix` `docs` `chore` `refactor` `test`

Mentor's closing note: You have everything here to go from zero to a finished, demo-ready project in 12 focused days. Don't read ahead and don't rush — the magic is in doing one green  at a time. When you finish a step, tell me, or ask me the  question next to it. I'm your mentor for the whole ride. **Start with Step 1. Go.**